Web Technologies

Web applications: other architectural approaches

from **SPA**, **PWA**, and **mash-up** to **CORS** and **JAMstack**

Prof. Sabin Corneliu Buraga – profs.info.uaic.ro/sabin.buraga/

Assoc. Prof. Andrei Panu – profs.info.uaic.ro/andrei.panu/

„The manner of giving is worth more than the gift.”

Pierre Corneille

(re)discover...

- what does SPA (Single-Page Application) represent
- what is PWA (Progressive Web Application)
- how can we develop Web *mash-ups*
- how to access resources via JavaScript through CORS (Cross-Origin Resource Sharing)
- what does JAMstack mean

advanced

The existence of (micro-)services and REST/GraphQL APIs – eventually, asynchronously invoked via Ajax – offer the premises for developing the following kind of Web apps:

🚿 **SPA** (Single-Page Application)

🏗️ **PWA** (Progressive Web Application)

🍛 **hybrid** (Web mash-up)

🎮 **Web game** – e.g., MMORPG
(Massively Multiplayer Online Role-Playing Game)

advanced

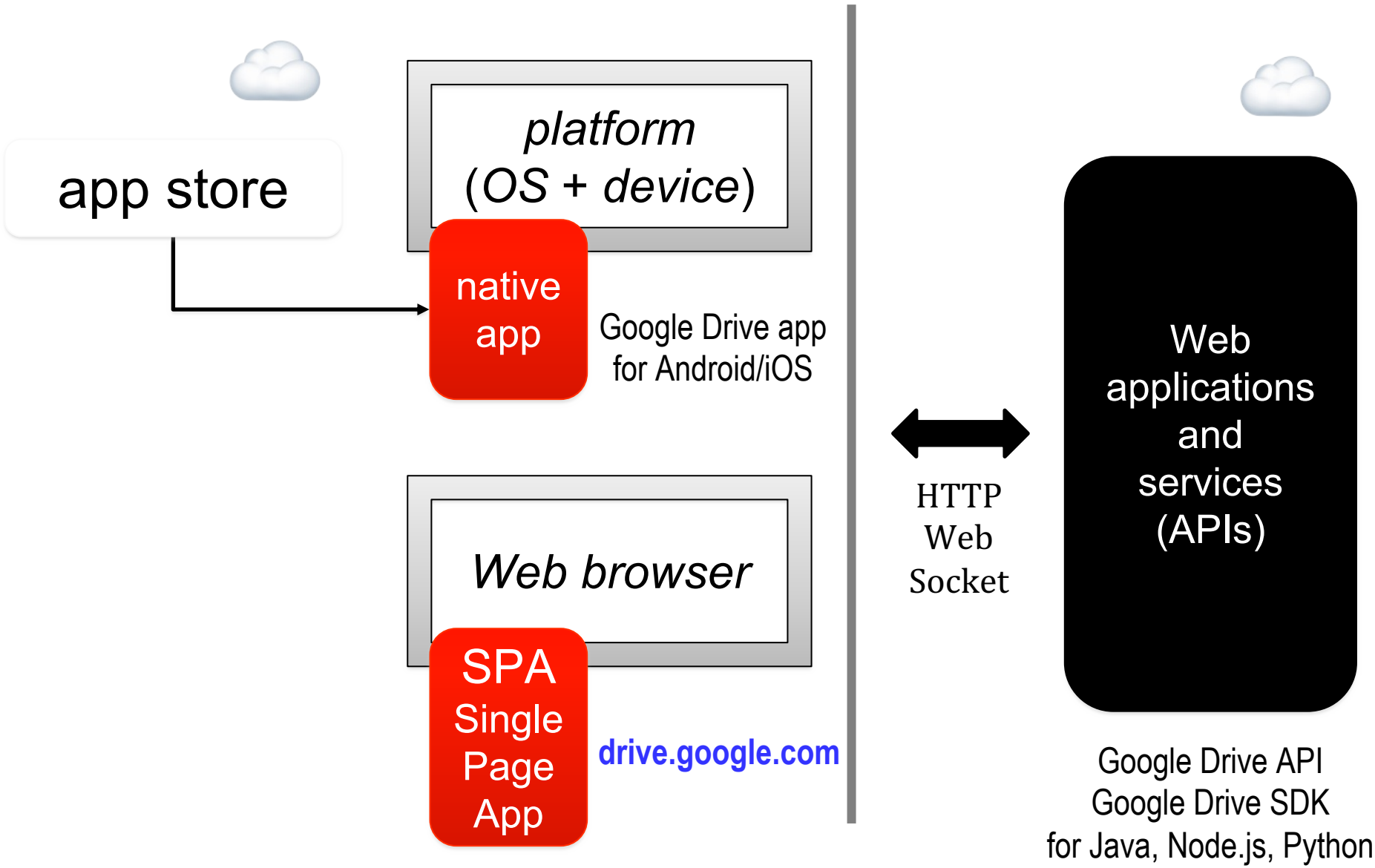
🚿 SPA (Single-Page Application)

advanced

Web interaction: spa

Dynamic rewriting of Web page content
– based on asynchronous data transfer –
following user interaction

advanced



adapted from Adrian Colyer

advanced

Web interaction: spa

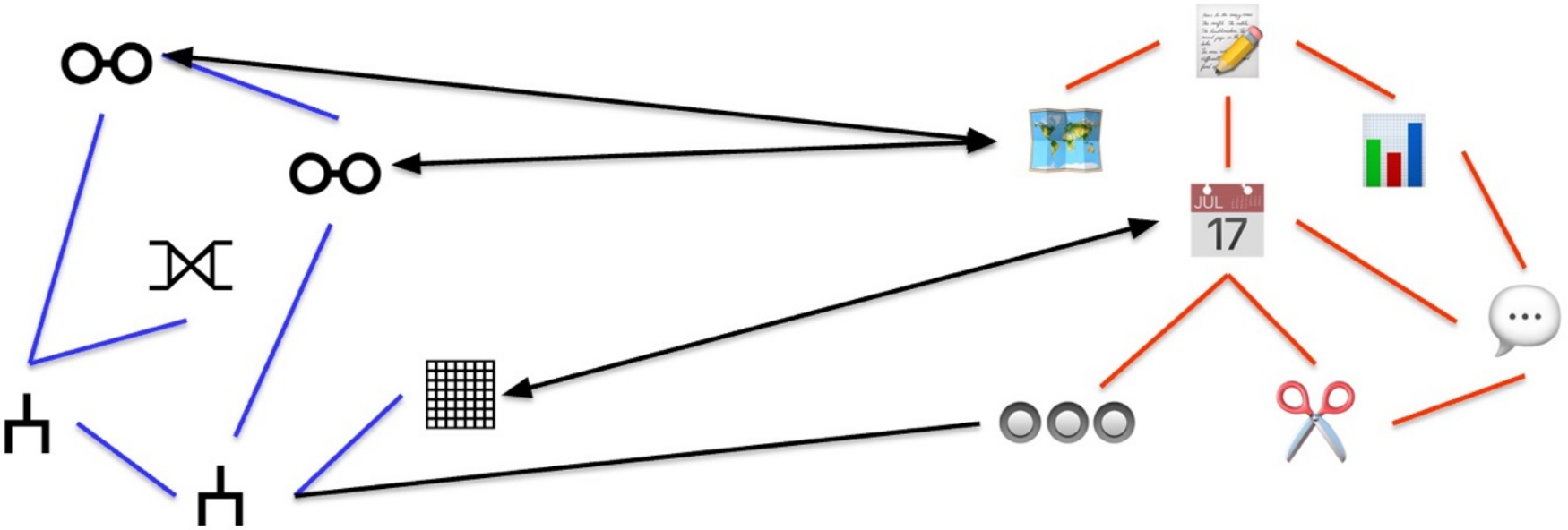
The application will be available at the same base URL, using a single HTML document, the content being rendered in the same Web browser window

in contrast with traditional approaches:
MPA (Multi-Page Application)

advanced

data model

interactive components



at the front-end level, each interactive component (e.g., button, player, edit field, calendar) may be in a certain state (*isPressed*, *isPlaying*, *isTyping*,…)

this state is not necessarily dependent on the back-end

advanced

Web interaction: spa

Problem:
efficient content rendering/refreshing

all the interactive components
– usually, in large number and more complex than those offered by HTML –
must be synchronized

advanced

Web interaction: spa

DOM alteration operations (mutation) are expensive for complex Web applications

e.g., a collaborative application (Canva, Figma, Google Docs, OverLeaf) or a Web game – for example, see the list **www.mmobomb.com/browsergames/mmorpg**

advanced

Web interaction: spa

Necessity:
Managing the global application state
at the Web browser level

advanced

Web interaction: spa

Solution:
application state management at runtime

URL ↔ state

advanced

Web interaction: spa

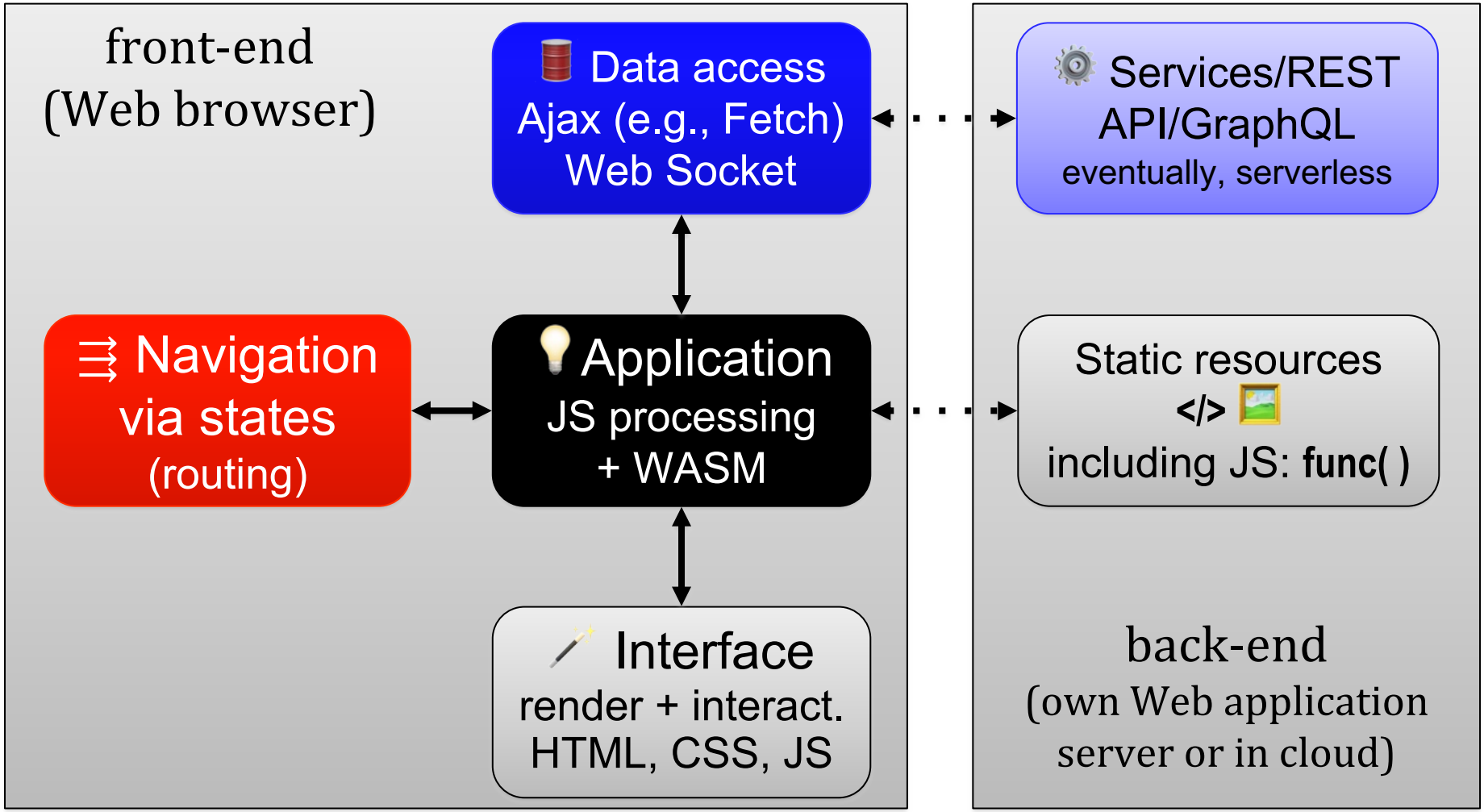
The current state of the view
is specified by the URL

usually, **/path/state** or **#state**

real example: Google Drive – states designated by URLs:
/drive/my-drive /drive/recent /drive/shared-with-me /drive/starred

advanced

Major components of a SPA
(adapted from Patrick Ackerman)



advanced

Web interaction: spa

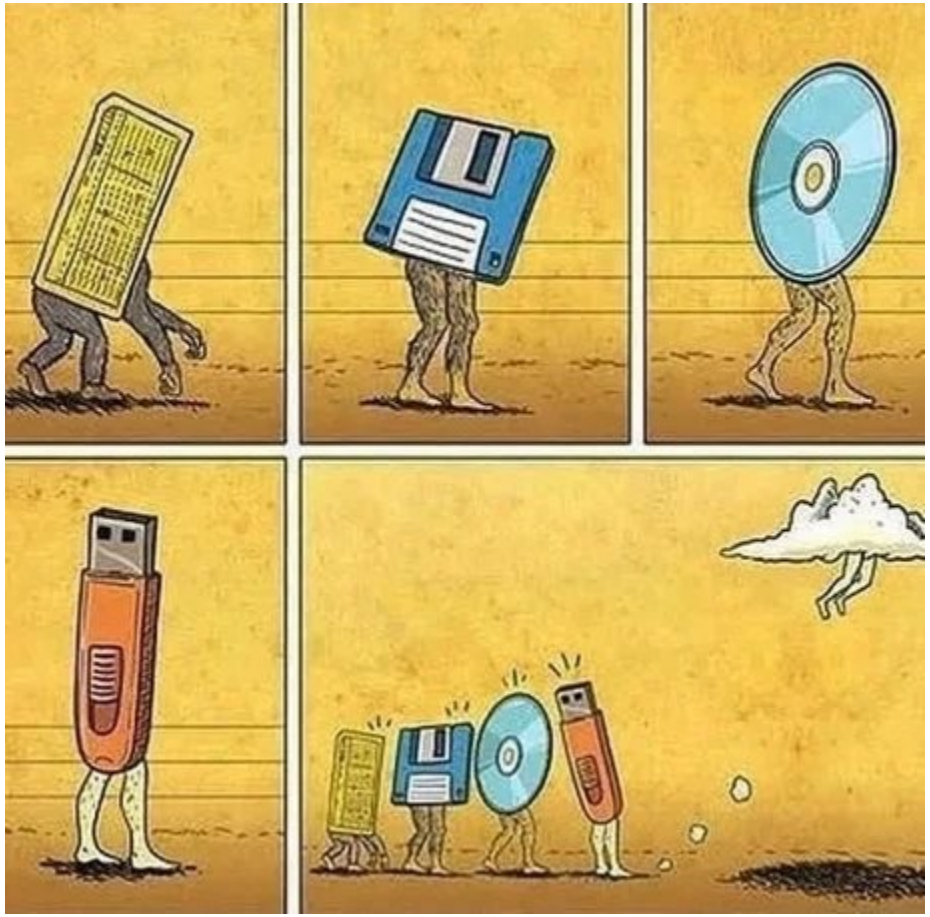
Usually, based on the
reactive (functional) programming paradigm
(FRP – functional reactive programming)

targets asynchronously processed data streams
(event stream, observable)

real case studies + resources:
github.com/stoeffel/awesome-fp-js
www.infoq.com/reactive-programming/

advanced

PWA (Progressive Web Application)



advanced

pwa

Web applications capable of providing functionalities adapted to the execution context

available on various platforms at Web browser level

use various APIs provided by the browser

html-now.github.io

webstatus.dev

advanced

pwa

Web applications capable of providing functionalities adapted to the execution context

resources of interest:

developer.mozilla.org/Web/Progressive_web_apps

web.dev/explore/progressive-web-apps

www.pwastats.com

advanced

pwa – characteristics

Adaptation to the device

review the discussions on
responsive (Web) design

developer.mozilla.org/Learn_web_development/Core/CSS_layout/Responsive_Design

advanced

pwa – characteristics

Interactions similar to native applications that are executed on desktop/mobile devices

advanced

pwa – characteristics

Offer support for caching: offline vs. online

HTML APIs provided by the browser are used:
Web Workers – non-interactive JavaScript programs executed in background, which do not have access to the DOM tree and are independent of other JS scripts

developer.mozilla.org/Web/API/Web_Workers_API/Using_web_workers

advanced

pwa – characteristics

Offer support for caching: offline vs. online

HTML APIs provided by the browser are used:
Web Workers – non-interactive JavaScript programs executed in background, which do not have access to the DOM tree and are independent of other JS scripts

the execution environment of a worker is completely separated, the code being run concurrently/in parallel, asynchronously

advanced

pwa – characteristics

Offer support for caching: offline vs. online

HTML APIs provided by the browser are used:
Web Workers – non-interactive JavaScript programs executed in background, which do not have access to the DOM tree and are independent of other JS scripts

the resources provided by external services can be asynchronously loaded via HTTPS by JS programs of type service workers

www.w3.org/TR/service-workers/

advanced

pwa – characteristics

Offer support for caching: offline vs. online

"Service workers essentially act as proxy servers that sit between Web applications, and the browser and the network (when available). They are intended to (amongst other things) enable the creation of effective offline experiences, intercepting network requests, and taking appropriate action based on whether the network is available, and updated assets reside on the server."
developer.mozilla.org/Web/API/Service_Worker_API

advanced

pwa – characteristics

Data will be transferred “securely” (with HTTPS) from a “trusted” source

advanced

pwa – characteristics

The content can be permanently refreshed, retrieved via Web (micro-)services

advanced

pwa – characteristics

The PWA app can be installed
– e.g., eventually, using an app store –
and shared

Web mash-ups

mash-ups

Hybrid Web applications – mash-ups

Combination – at client-level and/or server-level –
of content from multiple data sources,
retrieved (a)synchronously,
providing a new functionality/experience

mash-ups

Example:

we want to create an application that provides music information depending on the user's physical activity, based on some public Web services

mash-ups

www.last.fm/api/rest

+

dev.fitbit.com/build/

LaFi
hybrid Web application

API ROOT

The API root URL is located at http://ws.audioscrobbler.com/2.0/

Generally speaking, you will send a method parameter expressed as 'package.method' along with method specific arguments to the root URL. The following parameters are required for all calls:

api_key : A Last.fm API Key.
method : An API method expressed as *package.method*, corresponding to a documented last.fm API method name.

For example:

```
http://ws.audioscrobbler.com/2.0/?method=artist.getSimilar&api_key=xxx...
```

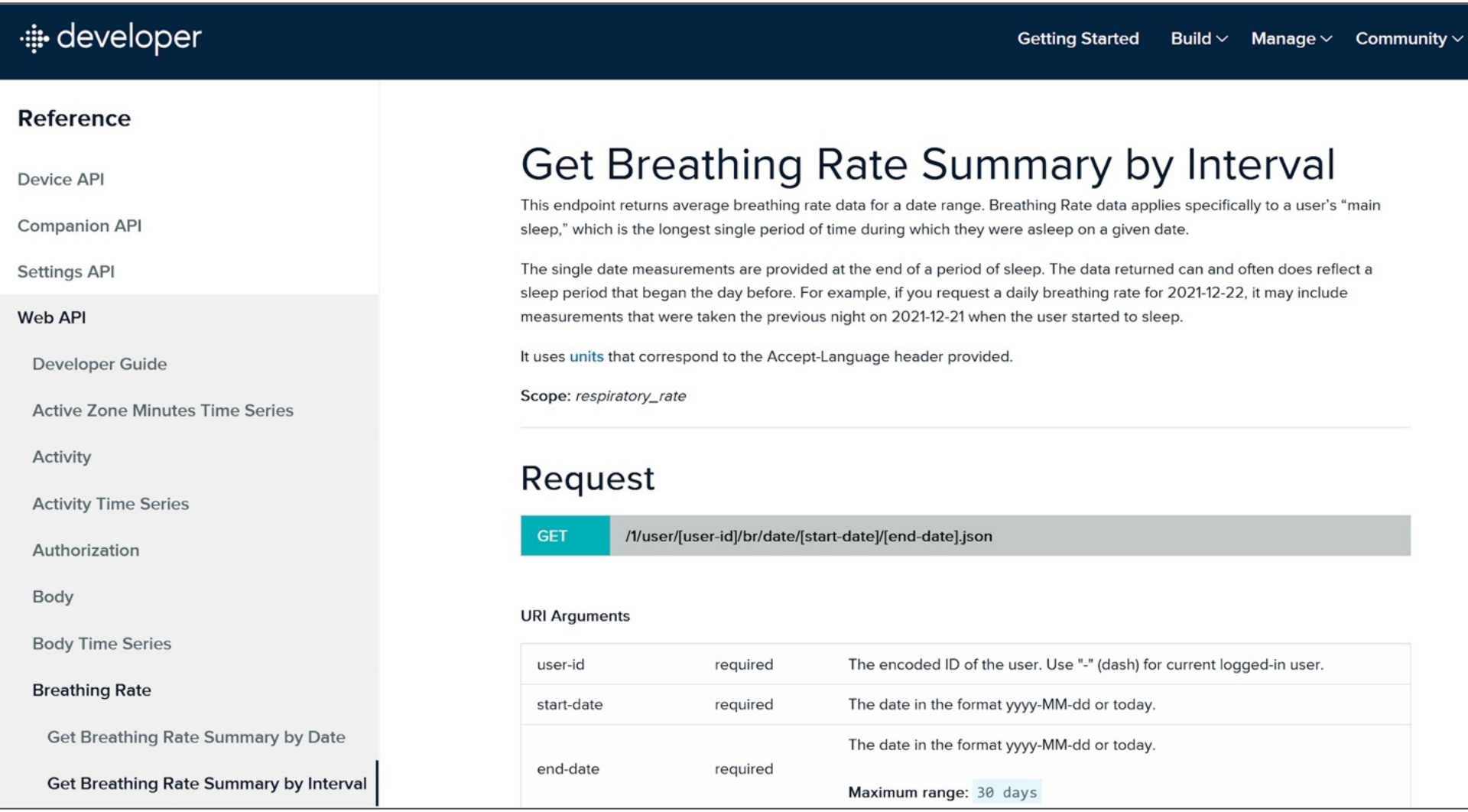
If you are accessing a *write* service, you will need to submit your request as an HTTP POST request. All POST requests should be made to the root url:

```
http://ws.audioscrobbler.com/2.0/
```

With all parameters (including the 'method') sent in the POST body. In order to perform write requests you will need to authenticate a user with the API. See authentication for more.

acces to Last.fm services via an authentication key

mash-ups



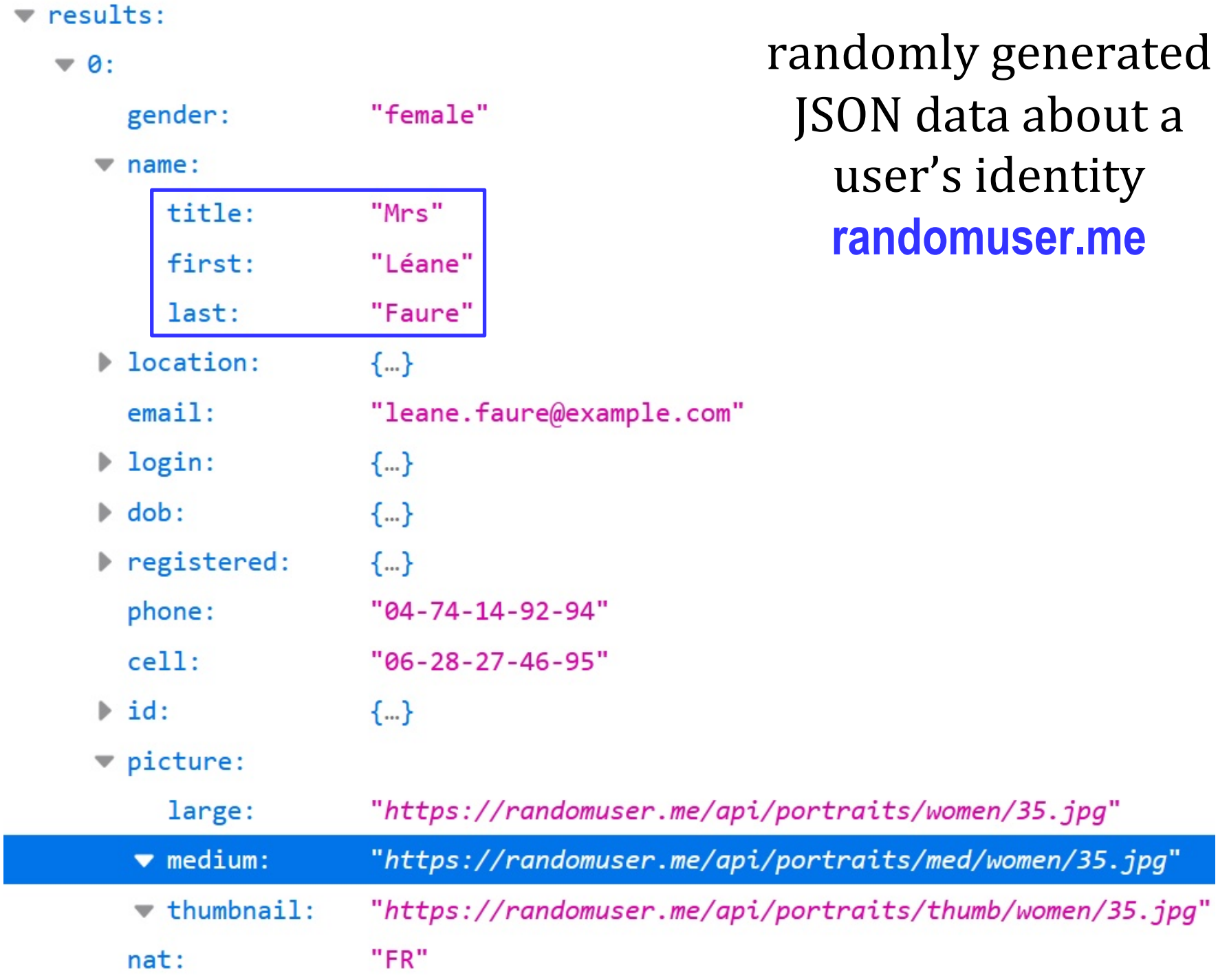
FitBit's REST API provides data in JSON format

(instead of) break

case study: HuDo

Creating a client-level Web mash-up
HuDo (Humans ‘n’ Dogs)

dog images via **dog.ceo/dog-api/** – JSON response:
{ "status": "success", "message": “*image_URL*" }
+
random data about users: **randomuser.me/api/**
complex JSON response



randomly generated JSON data about a user's identity

randomuser.me

HTML

```
<div id="images"><span id="human"></span> vs <span id="dog"></span></div>
<div id="user-name"><span id="text"></span></div>
```

CSS

```
img {
  height: 100px;
}
#user-name {
  font-family: sans-serif;
}
```

JavaScript

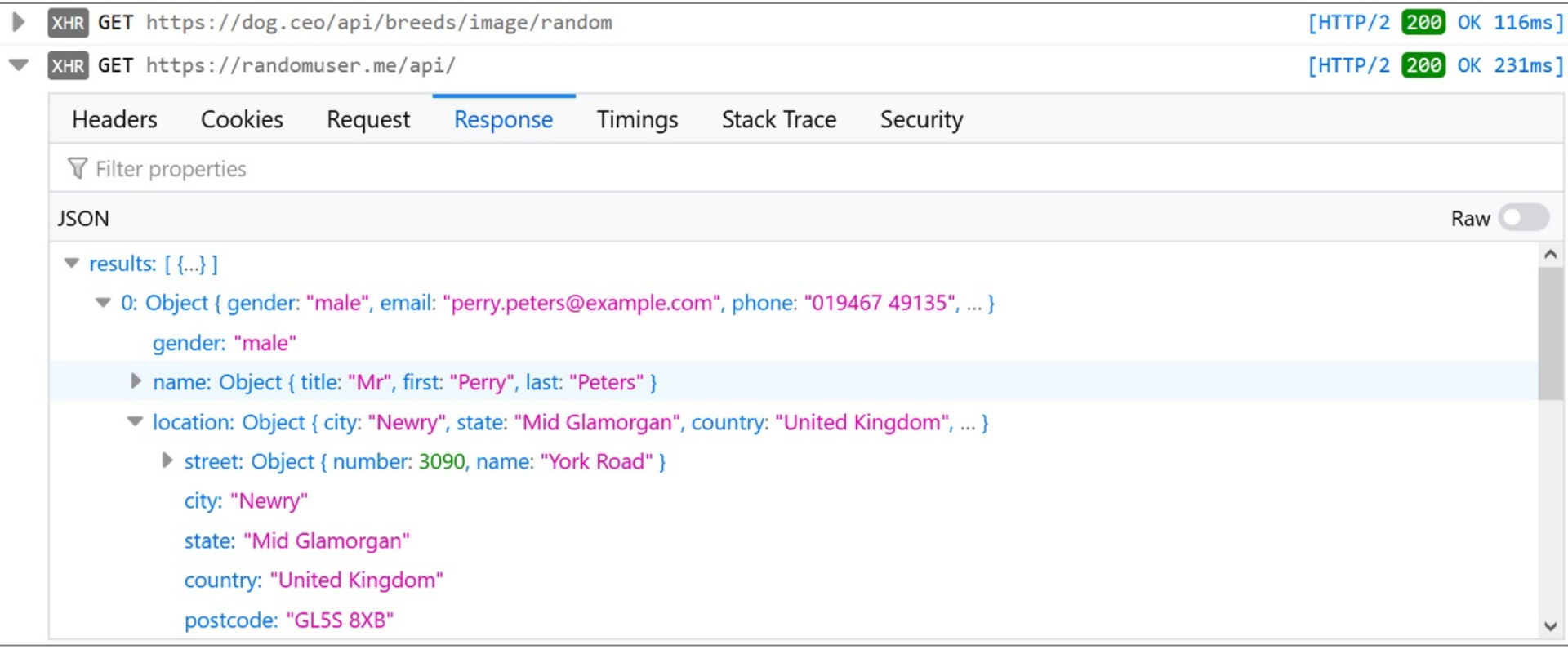
```
// verificăm dacă avem succes (codul de stare e 2XX)
// pe baza unei promisiuni
function status(response) {
  if (response.status ≥ 200 && response.status < 300) {
    // cererea poate fi rezolvată
    return Promise.resolve(response)
  } else {
    // cererea a fost rejectată
    return Promise.reject(new Error(response.statusTex
  }
}

// generăm în cadrul unui element identificat prin 'id
// elementul <img /> pentru a include URL-ul fotografi
// (parametrul 'url' dat funcției) ce reprezintă un an
function genImg(url, id) {
  const elem = document.createElement('img');
  elem.setAttribute('src', url);
  document.getElementById(id).appendChild(elem);
}

// generăm în cadrul unui element identificat prin 'id
// elementul <div> pentru a include un sir de caractere
// (parametrul 'text' dat funcției)
function genDiv(text, id) {
  const elem = document.createElement('div');
  elem.textContent = text;
  document.getElementById(id).appendChild(elem);
}
```


vs


use two fetch() calls to get JSON data from the two Web services (public APIs)

the received URLs are used to generate, with DOM, the <img> corresponding elements

the complete source code at jsfiddle.net/busaco/hp2gxc78/



inspecting HTTP requests initiated by the program to get JSON responses from the two REST APIs

The HTML document in which data will be „injected” – transferred asynchronously via Fetch API – coming from the two Web services:

```
<div>
   <span id="human">
      <!– here the image of a person will be included -->
   </span>
   vs
   <span id="dog">
      <!– and here, the image of a dog -->
   </span>
</div>
<div id="user-name">
   <span id="text">
      <!– display the person’s identity -->
   </span>
</div>
```

```
// helper functions which modify the document's DOM tree

// generate inside an element identified by 'id'
// the <img /> element to include the URL of the photo
// (the 'url' parameter passed to the function) which represents an animal
function genImg (url, id) {
    const elem = document.createElement ('img');
    elem.setAttribute ('src', url);
    document.getElementById (id).appendChild (elem);
}

// generate inside an element identified by 'id'
// the <div> element to include a string
// (the 'text' parameter passed to the function)
function genDiv (text, id) {
    const elem = document.createElement ('div');
    elem.textContent = text;
    document.getElementById (id).appendChild (elem);
}
```

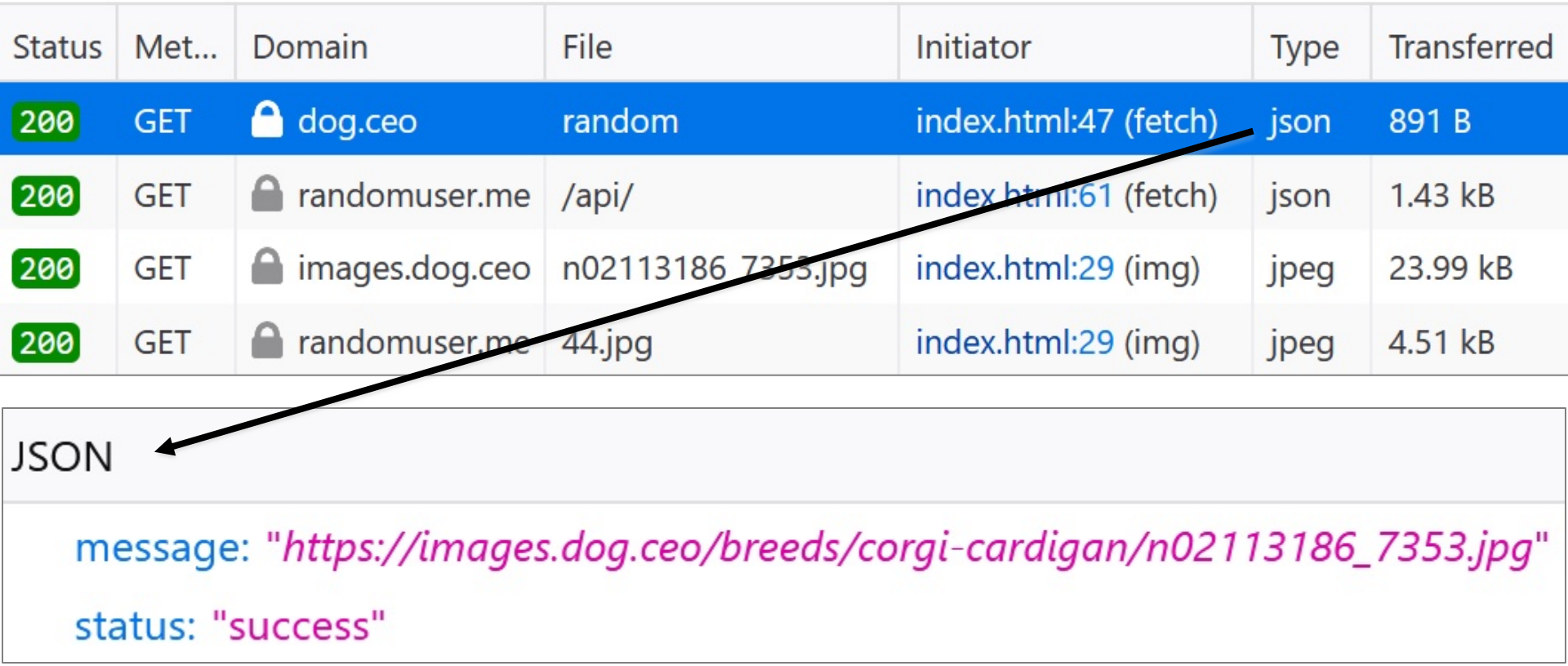
```
// check if successful (status code is 2XX)
function status(response) {
    if (response.status >= 200 && response.status < 300) {
      // the request can be resolved
      return Promise.resolve (response)
    } else {
      // the request was rejected
      return Promise.reject (new Error (response.statusText))
    }
}
```

```
const URLDOGS = 'https://dog.ceo/api/breeds/image/random';

// use Dog API to obtain a random image of a dog
fetch (URLDOGS)
  .then (status)        // was the data received successfully?
  .then (response => response.json ())
  .then (json => {
    // process received JSON data...
    // the 'message' property stores the URL of the dog's image
    genImg (json.message, 'dog');
  })
  .catch (error => {    // display the occurred error
    document.getElementById ('dog').textContent = error;
  });
```

```
const URLUSER = 'https://randomuser.me/api/';

// obtain data about a fictive user algorithmically generated
fetch (URLUSER)
  .then (status)        // was the data received successfully?
  .then (response => response.json ())
  .then (json => {
    // process received JSON data…
       // get the URL of the fictive person's image
    genImg (json.results[0].picture.medium, 'human');
       // obtain the identity
    genDiv (json.results[0].name.title + ' ' +
            json.results[0].name.first+ ' ' + json.results[0].name.last, 'text');
  })
  .catch (error => {    // display the occurred error
    document.getElementById ('human').textContent = error;
  });
```

Status	Met...	Domain	File	Initiator	Type	Transferred
200	GET	dog.ceo	random	index.html:47 (fetch)	json	891 B
200	GET	randomuser.me	/api/	index.html:61 (fetch)	json	1.43 kB
200	GET	images.dog.ceo	n02113186_7353.jpg	index.html:29 (img)	jpeg	23.99 kB
200	GET	randomuser.me	44.jpg	index.html:29 (img)	jpeg	4.51 kB

asynchronous data transfers (JSON + images)



asynchronous data transfers (JSON + images)

Status	Met...	Domain	File	Initiator	Type	Transferred
200	GET	dog.ceo	random	index.html:47 (fetch)	json	891 B
200	GET	randomuser.me	/api/	index.html:61 (fetch)	json	1.43 kB
200	GET	images.dog.ceo	n02113186_7353.jpg	index.html:29 (img)	jpeg	23.99 kB
200	GET	randomuser.me	44.jpg	index.html:29 (img)	jpeg	4.51 kB

JSON

message: "https://images.dog.ceo/breeds/corgi-cardigan/n02113186_7353.jpg"

status: "success"

GET https://images.dog.ceo/breeds/corgi-cardigan/n02113186_7353.jpg

Status 200 OK

Version HTTP/3

Transferred 23.99 kB (23.05 kB size)

Name: n02113186_7353.jpg
Dimensions: 500 × 375
MIME Type: image/jpeg

asynchronous data transfers (JSON + images)

case study: DoCa

Similarly, a Web mash-up at client level
DoCa (*Dogs & Cats*)

dog images (dalmatians) via **dog.ceo/dog-api/**
+
pictures – here, animanted GIFs – with cats:
Cat as a service (CATAAS) – **cataas.com/doc.html**
also study the OpenAPI specification: **cataas.com/doc.json**

HTML

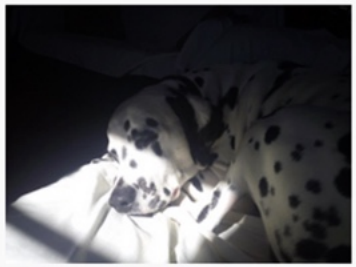
```
<div><span id="dog"></span> vs <span id="cat"></span></div>

```

CSS

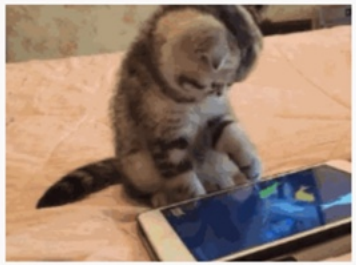
```
img {
  width: 200px;
}

```

JavaScript

```
  .then(response => response.json())
  .then(json => {
    // procesăm datele JSON primite...
    // proprietatea 'message' stochează URL-ul imaginii câinelui
    genImg(json.message, 'dog');
  })
  .catch(error => { // redăm eroarea survenită
    document.getElementById('dog').textContent = error;
  });

// invocăm random.cat API pentru a obține
// fotografia unei/unor pisici
fetch(URLCATS)
  .then(status) // datele au fost recepționate cu succes?
  .then(response => response.json())
  .then(json => {
    // procesăm datele JSON primite...
    // proprietatea 'url' stochează URL-ul fotografiei cu pisici
    genImg(json.url, 'cat');
  })
  .catch(error => { // redăm eroarea survenită
    document.getElementById('cat').textContent = error;
  });
```



vs



All HTML CSS JS XHR Fonts Images Media WS Oth

St...	Met...	File	Initiator	Transferred	Size
200	GET	cooper2.jpg	img	cached	181,89 kB
200	GET	gif?json=true	/busaco/z2...	622 B	183 B
200	GET	random	/busaco/z2...	899 B	89 B
200	GET	rDKy05t3ROh	img	1,07 MB	1,07 MB

4 requests | 1,25 MB / 1,07 MB transferred | Finish: 1,28 s

Headers Cookies Request Response Timings

Filter properties

JSON Raw

```
id: "rDKy05t3ROhKsT3P"
tags: (3)[ "kitten", "play", "gif" ]
created_at: "2022-05-01T21:53:01.498Z"
url: "https://cataas.com/cat/rDKy05t3ROhKsT3P?position=center"
mimetype: "image/gif"
```

Console

complete source code at jsfiddle.net/busaco/z2f3vp4m/

mash-ups

Based on RSS/Atom news feeds,
Web services, public APIs,…

SaaS (Software As A Service) „current”

implementation at the level of the:
Web client (browser) and/or Web server

mash-ups

Characteristics:

combination

visualization

aggregation

mash-ups

Combination

use of multiple (heterogeneous) data sources
can be multidimensional

example:
topic of interest + geographical location + time

a Web mash-up to study the effects of naval pollution

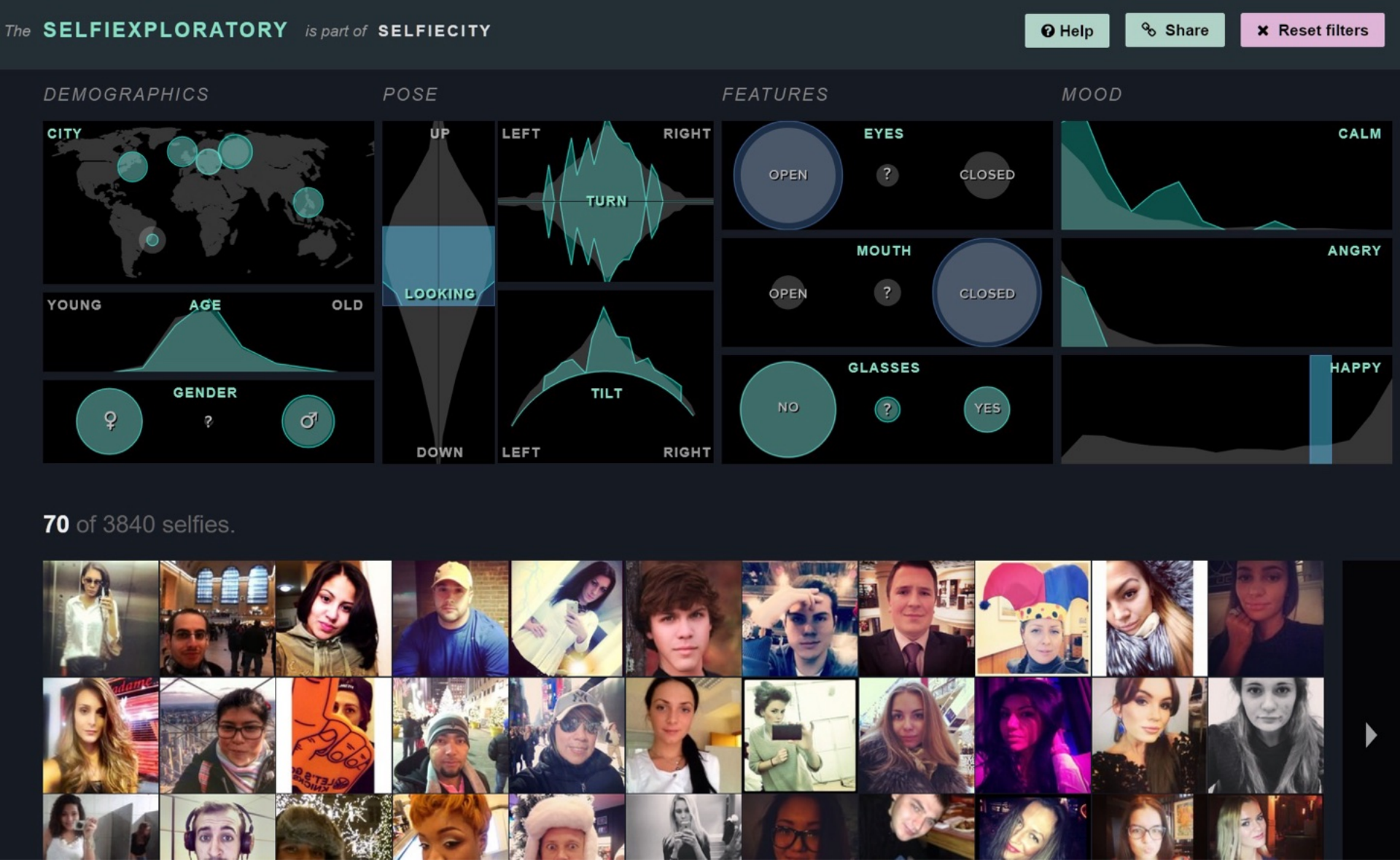
EU Shipping Emissions – **seafloor.pages.dev**

mash-ups

Aggregation

grouping and analyzing data from multiple sources: statistics, classifications, predictions, deductions,…

e.g., using machine/deep learning, “hidden” aspects of the processed data can be revealed



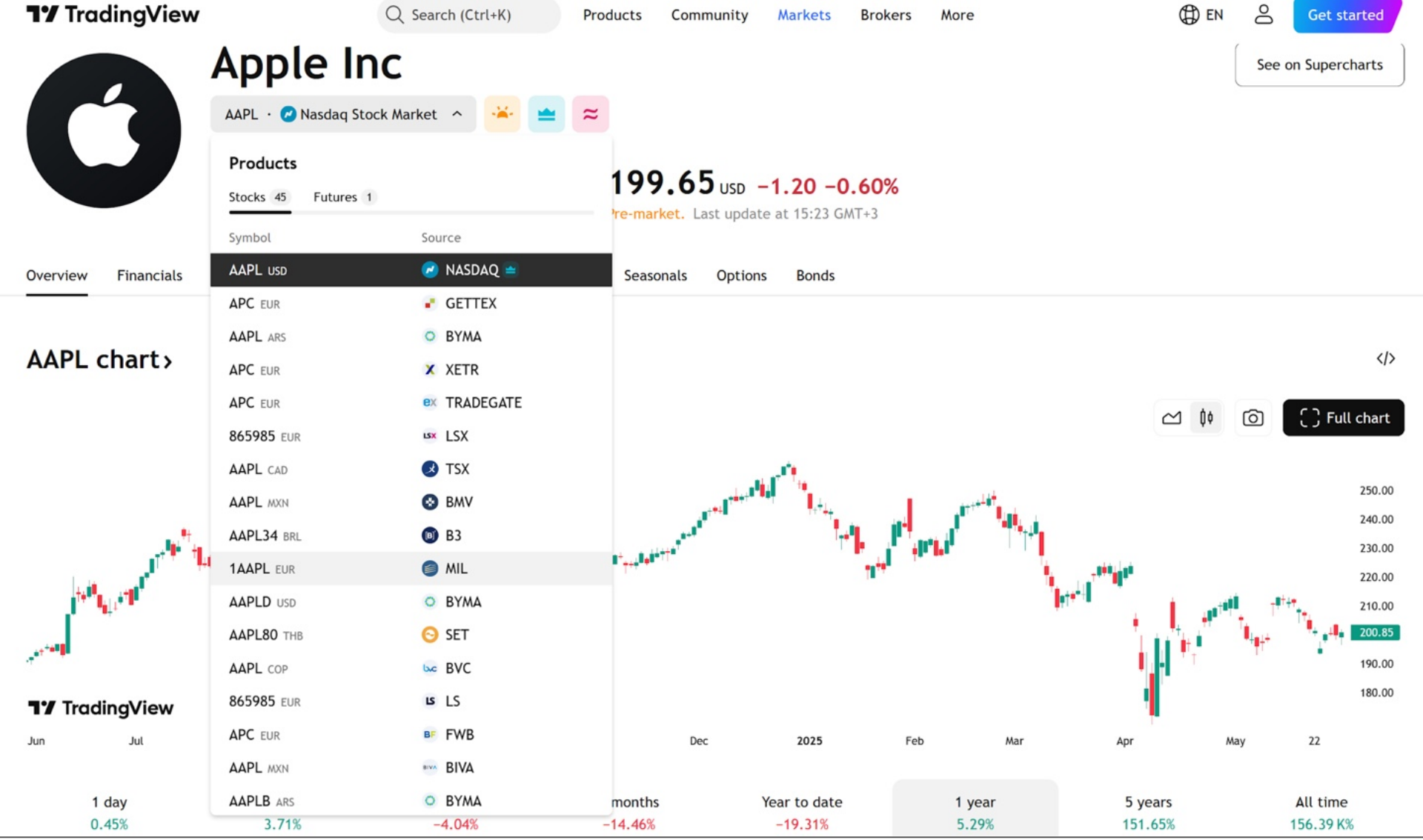
classification of images exposed by users according to age, posture, mood, and others

Selfiexploratory – selfiecity.net/selfiexploratory/

mash-ups

Visualization

Various visualization (presentation) techniques of aggregated data can be adopted:
charts, cartographic, cloud tags, three-dimensional,…



analyzing the evolution of a company's shares based on multiple data sources – **trandingview.com**

mash-ups

Data sources (data feeds)	JSON(-LD), RSS, geoRSS, HTML5 microdata, RDFa,…
Programming interfaces (APIs)	specific to public services and aiming at data processing
Libraries/frameworks for development	generic Web frameworks or offered by organizations
Interactive instruments (Web tools)	eventually, available in cloud
Platforms (Platform As A Service)	Digital Ocean, Heroku, Google Cloud Platform, MS Azure,…

Security issues regarding the access to resources, from the browser, via JavaScript?

advanced

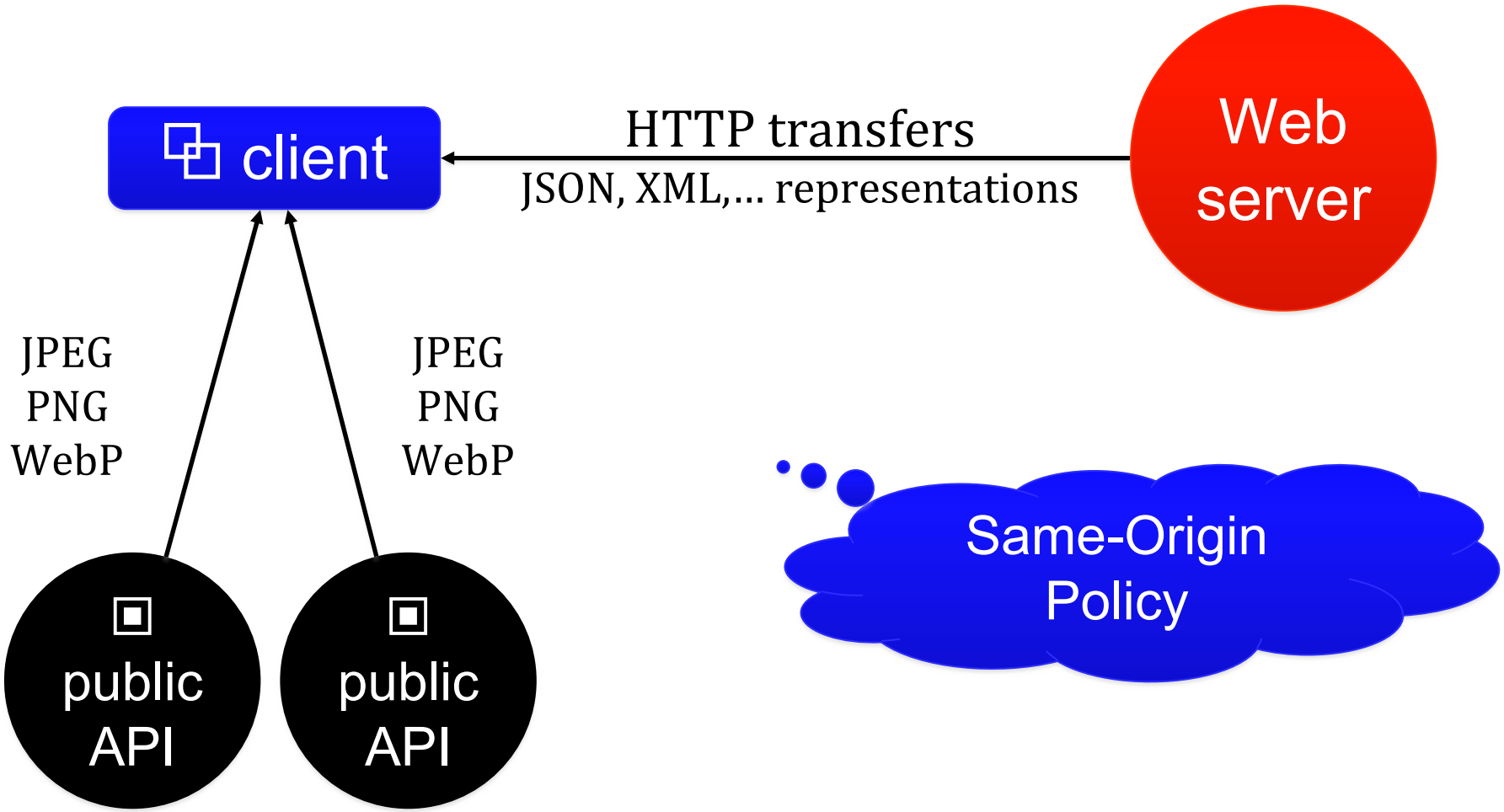
cors

Same-Origin Security Policy

"restricts how a document or script loaded from one origin can interact with a resource from another origin"

thus, a JavaScript program must access only data from the same origin – i.e., coming from the same Internet domain

advanced



only transfers involving resource representations regarding images (image/*), CSS files, and other JavaScript programs from the same origin are allowed

advanced

cors

Same-Origin Security Policy

prevents the cases when a loaded document/program from an origin can access/modify properties of a document belonging to another origin

developer.mozilla.org/Web/Security/Same-origin_policy

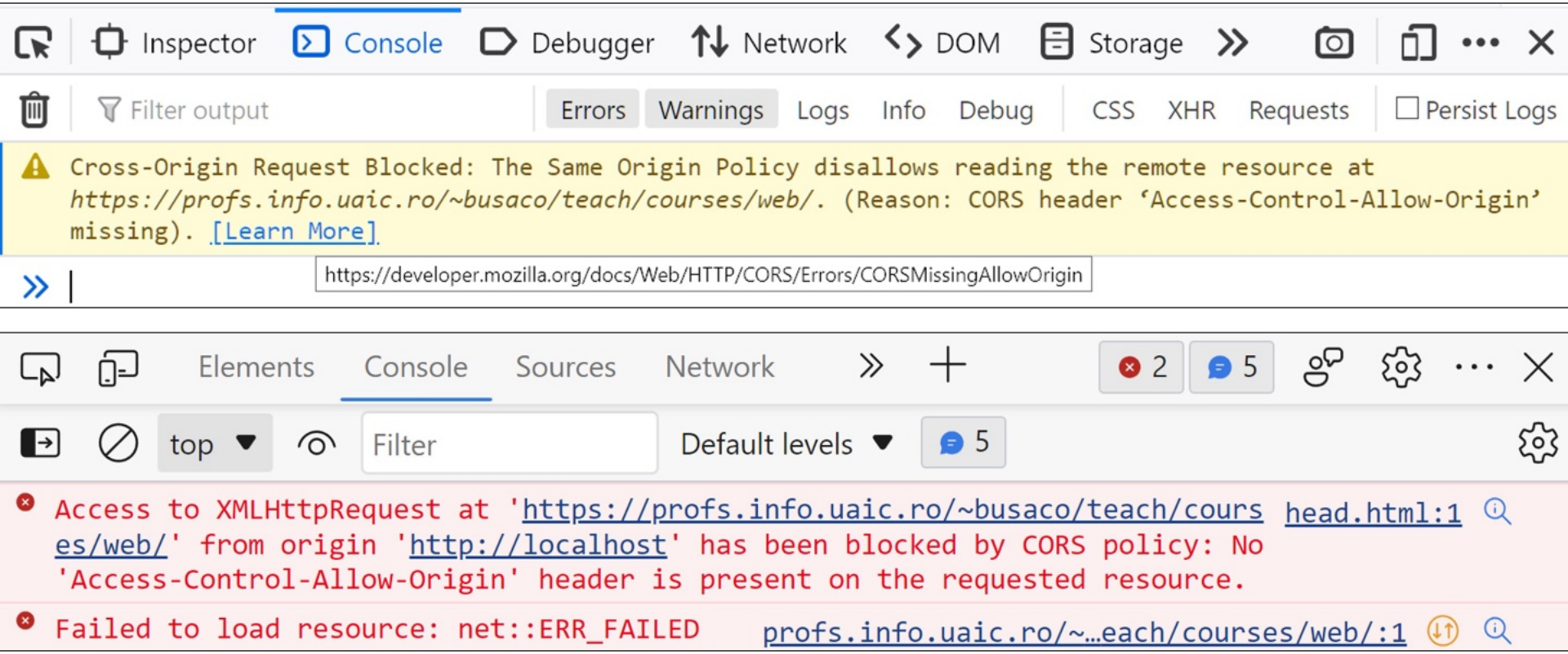
advanced

```
const URL = "https://profs.info.uaic.ro/~busaco/teach/courses/web/";

// send a HEAD request to obtain metadata about a resource
let client = new XMLHttpRequest ();
client.open ("HEAD", URL, true);
client.send ();
client.onreadystatechange = function () {
  // did we receive the header fields?
  if (client.readyState == 2) {
    // display the MIME type and the date of the last update
    alert (“The resource of type '" +
      client.getResponseHeader ("Content-Type") + "' was updated at " +
      client.getResponseHeader ("Last-Modified"));
  }
}
```

asynchronously retrieving, with **HEAD**, some metadata

advanced

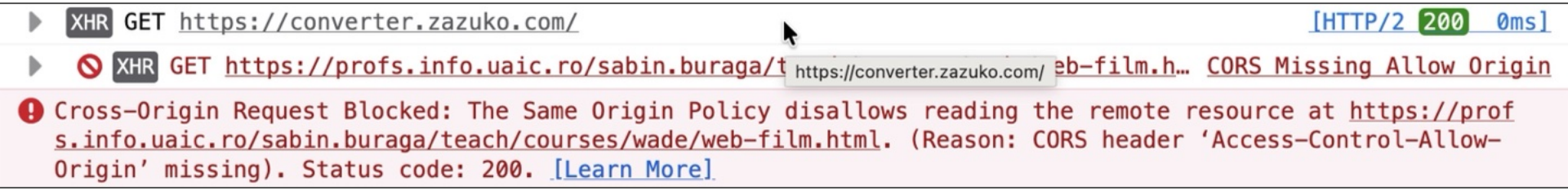
by trying to run in the browser the JS program sent by a Web server installed locally (localhost), we get:



URL of a different domain –> the Same Origin Policy is violated

advanced

real case: RDF Converter



advanced

cors

CORS (Cross-Origin Resource Sharing)

allows client-level sharing of resources from different origins (e.g., Internet domains)

thus, requests can be made between origins (cross-origin)

Fetch Living Standard - CORS protocol (08 May 2026)
fetch.spec.whatwg.org/#cors-protocol

advanced

cors

Resource sharing policies are based on certain fields in the HTTP message header

example (only GET requests are allowed, having in the header of the message specific fields from a client belonging to a given origin):

Access-Control-Request-Method: GET
Access-Control-Request-Headers: Content-Type, Accept
Origin: http://localhost:8000

advanced

cors

Resource sharing policies are based on certain fields in the HTTP message header

resources of interest:
developer.mozilla.org/en-US/docs/Web/HTTP/CORS

also study Steve Hobbs's tutorial (2019):
auth0.com/blog/cors-tutorial-a-guide-to-cross-origin-resource-sharing/

advanced

cors: real case #1

API providing jokes randomly generated
v2.jokeapi.dev/joke/Any?safe-mode
accepts GET requests sent from any client origin

Access-Control-Request-Method: GET
Access-Control-Allow-Origin: *

advanced

cors: real case #2

Google Play accepts POST requests sent only from the Google Drive service (the origin is **https://drive.google.com**) which include authorization header fields

Access-Control-Request-Method: **POST**
Origin: **https://drive.google.com**
Access-Control-Request-Headers: **authorization,x-goog-authuser**

advanced

cors: real case #3

Reddit.com allows POST and OPTIONS requests made from any origin, being accepted only certain header fields (access control depends on time)

Access-Control-Allow-Headers: **Content-Type,Origin,Accept,X-Signature**
Access-Control-Allow-Methods: **POST, OPTIONS**
Access-Control-Allow-Origin: *
Access-Control-Max-age: **1728000**

By using REST APIs/GraphQL, couldn’t we separate the way data is presented from the actual processing? What does JAMstack mean?

advanced

JAMstack

Modern approach for developing Web applications at the client-level

“A modern Web development architecture based on client-side JavaScript, reusable APIs, and prebuilt Markup.”
Mathias Biilmann

vision:
jamstack.org

advanced

JAMstack

Modern approach for developing Web applications at the client-level

JavaScript
the actual processing is done by the client
(usually, a Web browser) based on the features provided by the JS programming language and the related ecosystem

advanced

JAMstack

Modern approach for developing Web applications at the client-level

APIs
the functionalities and data are encapsulated by reusable APIs – own or offered by third parties – accessed (a)synchronously through HTTPS via JS programs

see previous courses:
REST, serverless, GraphQL

advanced

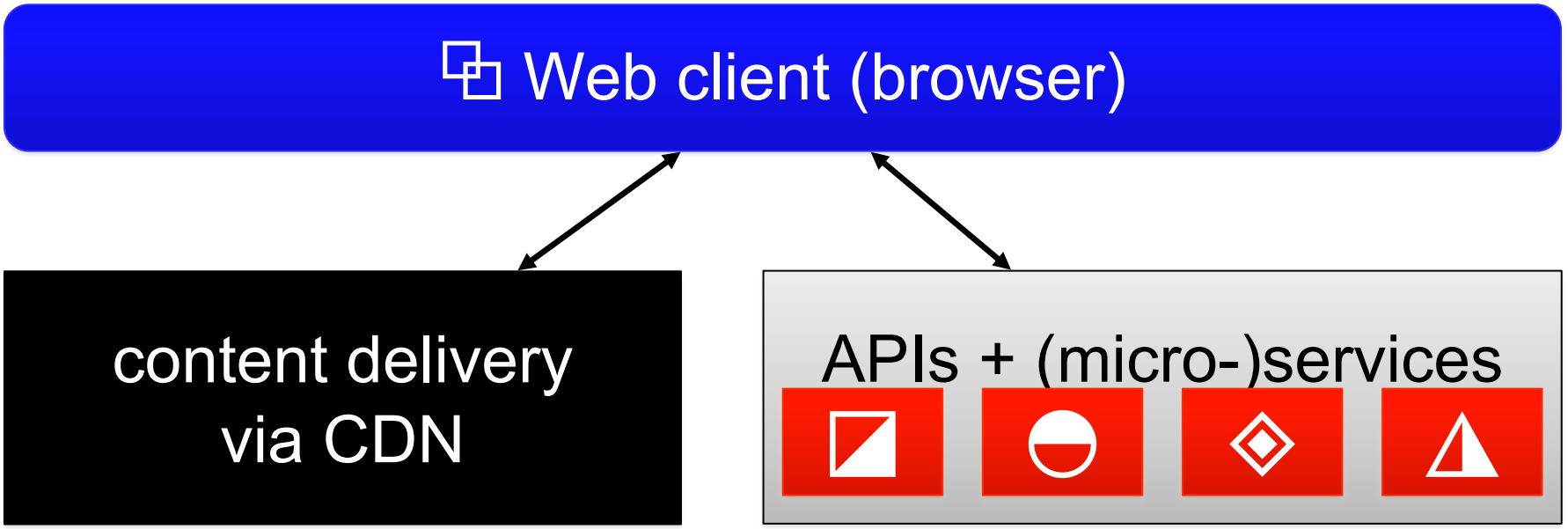
JAMstack

Modern approach for developing Web applications at the client-level

Mark-up
content delivery via templates defined by markup languages, pre-processed for presentation to the client (pre-rendering) and stored redundantly by CDN (Content Distribution Network), usually based on specific tools of type SSG (Static Site Generator) or headless CMS

advanced

JAMstack



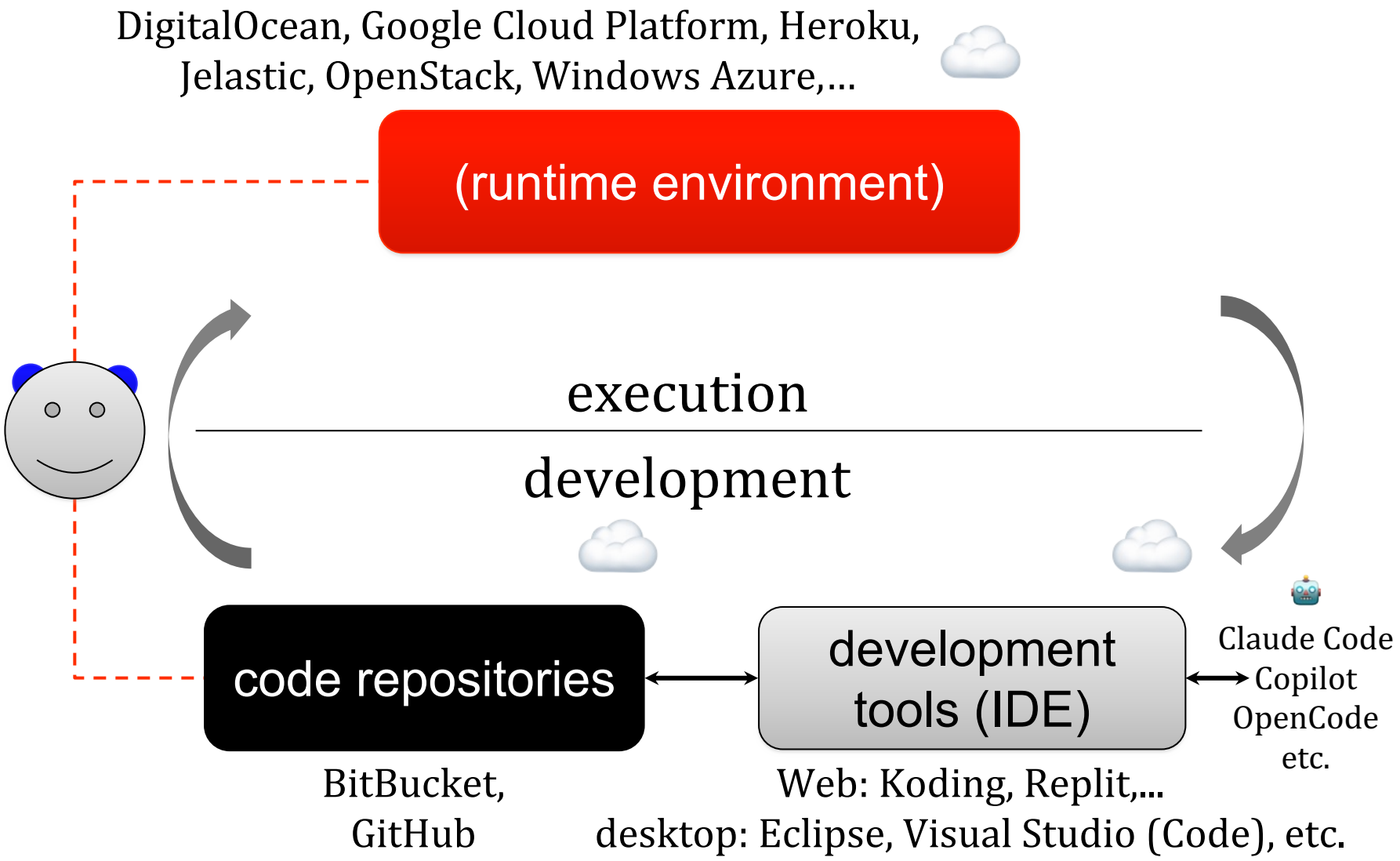
resources + tools:

github.com/automata/awesome-jamstack

also study Mathias Biilmann & Phil Hawksworth,
Modern Web Development on the Jamstack, O'Reilly, 2019

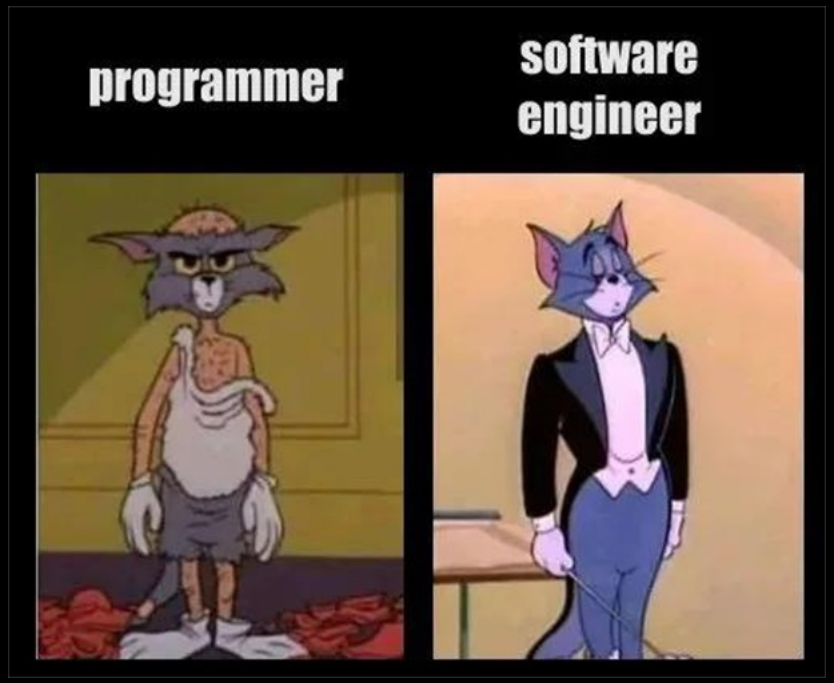
www.netlify.com/oreilly-jamstack/

Development as a Service



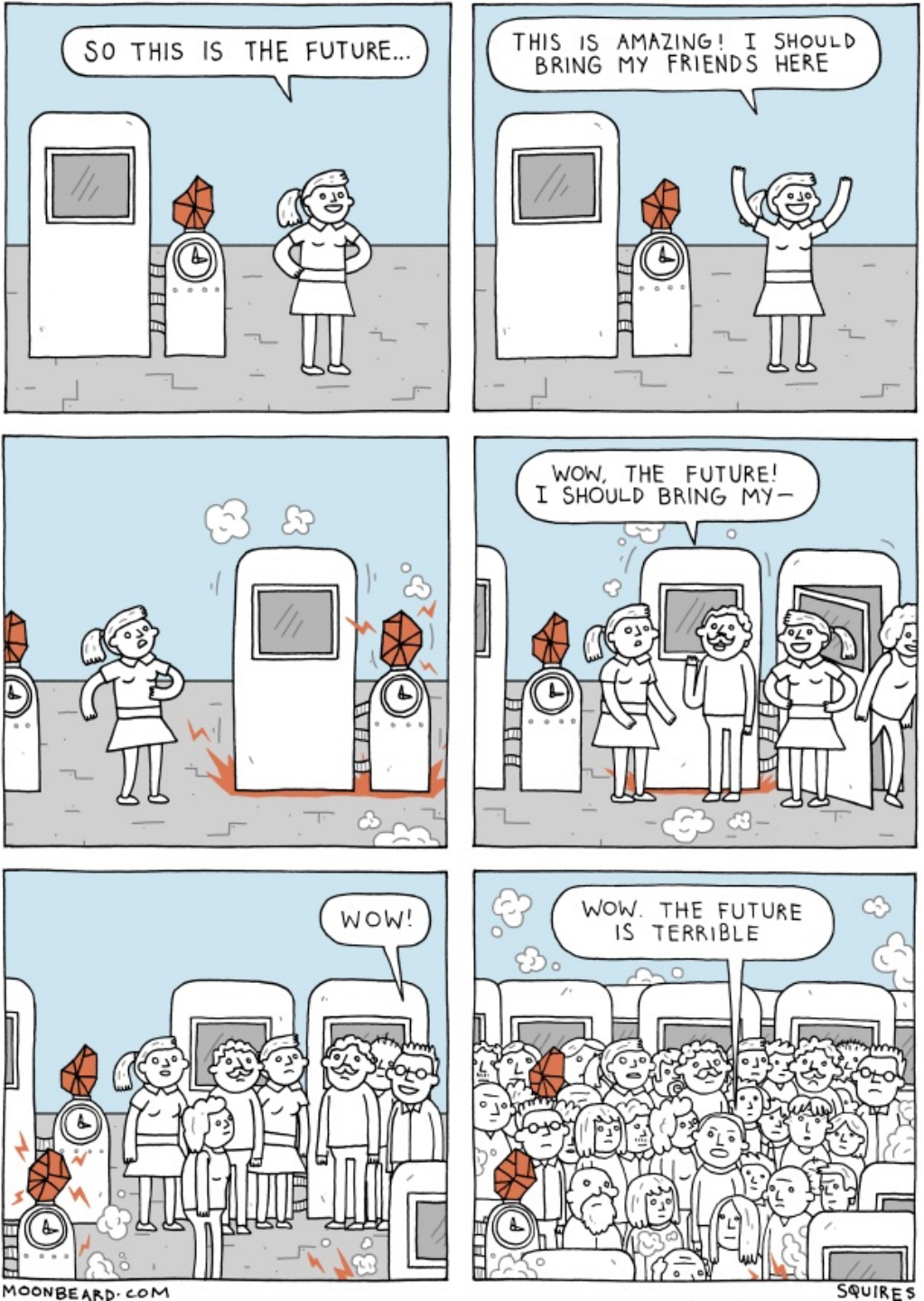
useful tools at **github.com/ripienaar/free-for-dev**

summary



other Web application architectures

from SPA and mash-ups to CORS and JAMstack


SO THIS IS THE FUTURE...
THIS IS AMAZING! I SHOULD BRING MY FRIENDS HERE
WOW, THE FUTURE! I SHOULD BRING MY—
WOW!
WOW. THE FUTURE IS TERRIBLE
MOONBEARD.COM
SQUIRES

Good luck!